

Electricity Sector Data Streaming and Analysis

COMP5339 Data Engineering — Assignment 2 (Group 156)

Member 1 (SID: 540989498) Member 2 (SID: 540929966) Member 3 (SID: 540903678)

May 2026

1 System Description

This project builds an end-to-end pipeline that retrieves real-time Australian electricity generation and emissions data from the OpenElectricity v4 REST API, replays it as an ordered event stream over the MQTT protocol, and renders it on a live map-based dashboard. The system is intentionally split into two cooperating Python processes that communicate only through a public MQTT broker. This mirrors a production publish/subscribe architecture and decouples data production from visualisation, so the dashboard can be restarted in the middle of a stream without losing already-persisted records and the publisher can replay history without re-fetching from the upstream API.

The publisher (`assignment2.ipynb`) is responsible for Tasks 1 through 4 and Task 6. It retrieves the per-facility power and emissions series for the NEM network, cleans and consolidates them into a single wide-format file, materialises that file alongside the optional market price and demand series, implements the Assignment 1 normalised DuckDB schema with two stream-oriented extensions, and continuously streams records to the broker. The subscriber (`assignment2_dashboard.ipynb`) is the Task 5 dashboard. It attaches to the same broker on a topic of our choosing, decodes each JSON message, updates a thread-safe rolling state, redraws a Plotly Dash UI every 1.5 seconds, and persists every received record back into the shared DuckDB schema. Storage is layered: raw JSON responses live as an immutable cache, the cleaned wide-format table is the single source of truth for replays, and DuckDB holds the analytical schema that both assignments share. Figure 1 summarises the end-to-end flow.

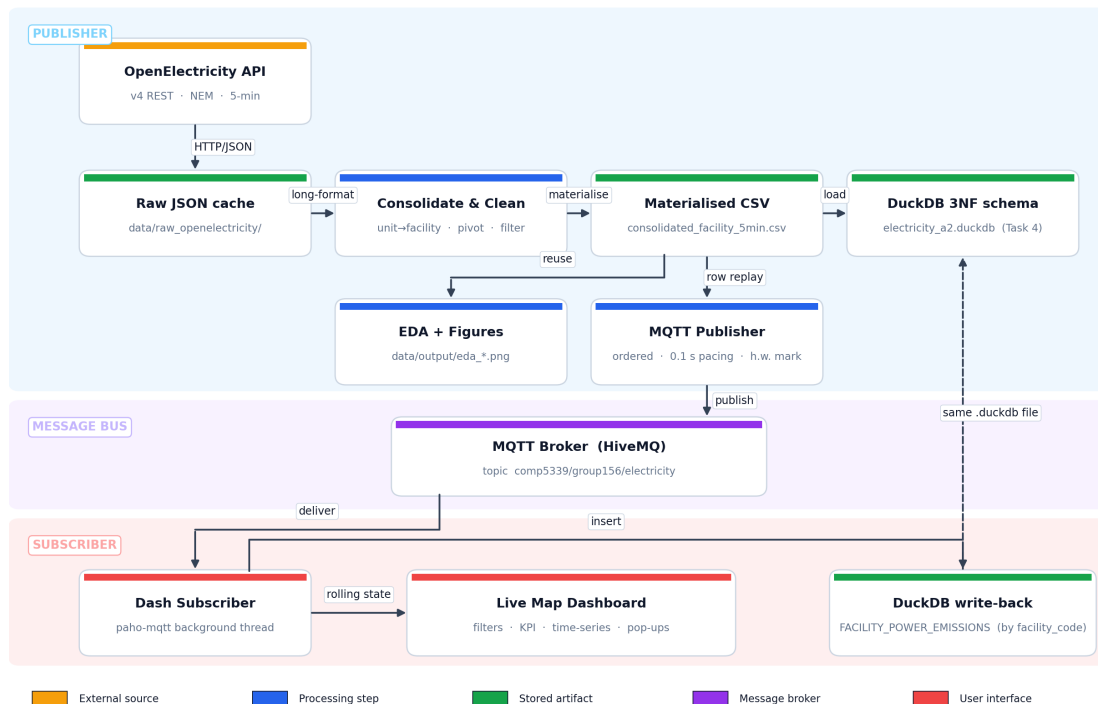


Figure 1: End-to-end system flow. The publisher (top lane) materialises the upstream API into a layered on-disk cache and streams records into the broker; the subscriber (bottom lane) consumes the topic and writes each received record back into the shared schema.

The component roles are summarised in Table 1. The split is deliberate. The broker provides ordered, backpressure-free delivery between two processes that need never share memory; the on-disk cache means the upstream API is touched at most once per week of data, regardless of how many times the stream is replayed; and the shared DuckDB file means the same normalised schema serves the historical NGER/CER/ABS data from Assignment 1 and the new 5-minute facility observations from Assignment 2.

Table 1: System components and responsibilities

Component	Role
Publisher notebook	Tasks 1–4, 6: retrieves the OpenElectricity series, cleans/consolidates to CSV, implements the Assignment 1 DuckDB schema, and streams records to MQTT continuously.
MQTT broker	Public HiveMQ broker at <code>broker.hivemq.com:1883</code> . Carries one JSON message per facility-interval on the topic <code>comp5339/group156/electricity</code> .
Dashboard notebook	Task 5: subscribes to the topic, maintains a rolling state on a background thread, draws a live Plotly Dash map and time-series charts, and writes received records into the shared DuckDB schema.
Storage layer	Layered cache: raw JSON (<code>data/raw_openelectricity/</code>), consolidated CSV (<code>consolidated_facility_5min.csv</code>), and analytical database (<code>electricity_a2.duckdb</code>).

2 Data Acquisition, Cleaning and Transformation

The OpenElectricity v4 REST API exposes two endpoints relevant to this project. `GET /v4/facilities/` returns the facility/unit metadata catalogue (location, fuel technology, registered capacity, status), and `GET /v4/data/facilities/{network}` returns the power and emissions time-series. A single 5-minute request is capped at an 8-day window, so the chosen 7-day window of 12–19 May 2026 fits in one pass per batch without pagination. Free-tier accounts have a daily allowance of 500 requests, so all interaction with the API is funnelled through a small `OpenElectricityClient` class that handles bearer authentication, ISO-8601 UTC date formatting, and a bounded retry policy — it retries only on transient 429 responses (honouring the `Retry-After` header) and 5xx errors, but fails fast on permanent 4xx errors so a bad request is never retried blindly. A `requests_made` counter makes the daily-budget usage observable at a glance.

To stay well under the 500-request budget the client batches facility codes. The data endpoint accepts just under 30 codes per call, so a batch size of 25 covers approximately 430 operating NEM facilities in roughly 18 data requests, with one extra facility-list request and one optional market request — about 20 calls in total. Every raw response is written to disk as JSON before parsing, and a cache check at the top of each batch makes re-runs a no-op. This is the materialisation that Task 2 requires: once a window has been retrieved the API is never touched again, no matter how often the consolidation step or the stream is replayed. Many NEM facilities comprise several generating units — for example Bayswater Power Station’s four coal turbines — and the API returns time-series keyed by `unit_code` without echoing the parent facility code. We rebuild the `unit` → `facility` mapping from the metadata and aggregate units to the facility level by summing values for each (`facility_code`, `interval_ts`) pair, which is correct for both metrics because power is additive across parallel units and emissions are additive by definition.

The cached batches are then flattened into a long-format (`interval_ts`, `facility_code`, `unit_code`, `metric`, `value`) table and progressively cleaned. Timestamps are normalised to UTC (the API returns network-local time) and strictly trimmed to the requested window; values are coerced to numeric and null or blank rows dropped; metric labels are standardised so that power becomes `power_mw` and emissions becomes `emissions_tco2e`; units are then summed up to facility level and the two metrics pivoted wide; rows missing *both* metrics are dropped as uninformative; and the wide table is finally enriched with facility metadata (network

region, dominant fuel technology, registered capacity, latitude and longitude) by joining on `facility_code`. The result is persisted as `consolidated_facility_5min.csv` — the single cache every downstream task reads from. The optional market price and demand series is fetched in one network-level call and stored separately in `market_price_demand_5min.csv`, joinable on `interval_ts`.

3 Exploratory Data Analysis

Before publishing the stream, we ran an exploratory pass over the consolidated week to verify that the cleaning pipeline left a complete analysis-ready dataset and to characterise the shape of the NEM under the May 2026 sample. The consolidated table contains **668,096 facility-interval rows across 353 operating facilities and all five NEM regions** (NSW1, QLD1, VIC1, SA1, TAS1) over the seven-day window — 353 of the 542 facilities in the OpenElectricity NEM catalogue produced measurable generation during the sampled week, the remainder being committed, retired or non-dispatching units carried for completeness. A missing-value audit returned exactly zero nulls across the ten columns, confirming that the unit-aggregation, metric-pivoting and drop-both-null steps in Task 2 collectively leave no gap in either metric. Power output spans a wide range (−14.7 to 2,718 MW with a median of 0.13 MW), which reflects the long tail of small renewable plants alongside a handful of multi-gigawatt coal stations; negative power values correspond to batteries actively charging from the grid, which we deliberately retain because they are physically meaningful.

The generation mix over the sampled week is sharply concentrated in two fuel categories (Figure 2, left). Black and brown coal together contribute **about 58%** of total energy (41.7% black, 16.1% brown), with wind at 15.9%, hydro at 8.2%, utility-scale solar at 7.5% and batteries at 4.8% making up almost all the remainder. The emissions picture, in contrast, is dramatically more skewed: the same two coal categories account for **95.4%** of all tCO₂e emitted (coal_black 63.0%, coal_brown 32.4%), with gas contributing roughly 4%. This concentration is the core analytical insight of the sampled week — emissions are not distributed across the fleet in proportion to output, they are produced by a small number of thermal generators that the dashboard’s power/emissions toggle can immediately isolate. The fleet-wide Pearson correlation between `power_mw` and `emissions_tco2e`, restricted to thermal plant, is $r = 0.985$ ($n = 127,028$), confirming that emissions within the thermal subset are an almost-linear function of output.

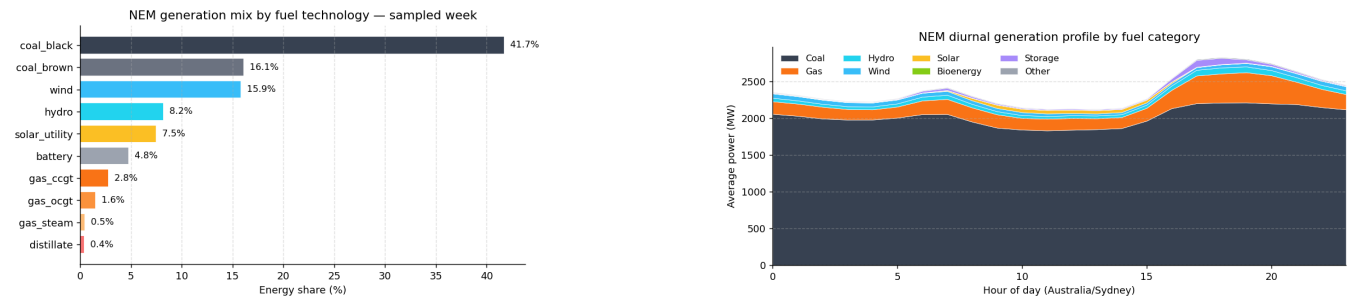


Figure 2: Left: NEM generation mix by fuel technology over the sampled week — coal dominates absolute energy but renewables make up close to a third of generation. Right: average diurnal profile by fuel category in Australia/Sydney local time — the solar peak between roughly 09:00 and 15:00 visibly displaces gas, while coal and wind provide a broadly stable baseline.

Regionally, NSW1 (1.20 TWh, 98 facilities) and QLD1 (1.09 TWh, 86 facilities) lead on energy, but Victoria carries the highest intensity at 0.77 tCO₂e/MWh because of its brown-coal fleet; South Australia and Tasmania, dominated by wind and hydro respectively, sit far below at 0.17 and 0.01 tCO₂e/MWh. The diurnal profile (Figure 2, right) further shows the solar peak between roughly 09:00 and 15:00 displacing gas during midday hours while coal provides a flat baseline — a pattern the dashboard’s stacked-area chart reproduces live as messages arrive.

4 Stream Publishing, Continuous Execution and Schema Integration

For each (`interval_ts`, `facility_code`) record the publisher emits a single JSON message containing everything the dashboard needs to render and persist it: facility code and name, network region, fuel technology, registered

capacity, latitude and longitude, the two metrics, the event time and a `published_at` timestamp for end-to-end latency measurement. Spec compliance is enforced explicitly. Records are sorted by (`interval_ts`, `facility_code`) before publishing so every subscriber observes events in the order they occurred; a `time.sleep(0.1)` follows every publish to honour the 0.1-second pacing requirement; and a high-water-mark file (`mqtt_last_publish.json` / `mqtt_continuous_state.json`) records the last event time streamed so that a re-run, or a new continuous round, publishes only records newer than the last, eliminating duplicates across restarts. A short self-subscribe sanity check publishes three messages and confirms they are received back, demonstrating the publish path independently of the dashboard.

For continuous execution (Task 6), Tasks 1–3 are wrapped in `run_continuous_stream()`, which replays the cached dataset *one 5-minute event-time slice per round* — the same slice the upstream API would return on each fresh retrieval in production — with a 60-second pause between rounds in addition to the 0.1-second pacing between messages. For each facility in the new slice, its current `power_mw` and `emissions_tco2e` are compared against the values from the previous interval, and a message is published only if at least one metric has changed; stable generators such as a coal plant at constant output overnight are silently skipped to avoid redundant broker traffic. On the first round, or after a wrap-around to the start of the cached week, every facility is treated as changed so the dashboard always starts from a complete snapshot. Setting `rounds=None` runs the stream unbounded until the kernel is interrupted, which is the production setting the brief describes; a small `MAX_ROUNDS` cap (default 2) is kept in the notebook purely so the demo cell finishes for a marker. A separate one-shot cell publishes a straight-through smoke test capped at five thousand messages, used only to verify the publish path independently of the continuous loop.

Task 4 re-implements the exact Assignment 1 3NF schema (`FUEL_TYPE`, `CORPORATION`, `FACILITY`, `NGER_REPORT`, `LSRE_STATION`, `ABS_REGION`, `ABS_INDICATOR`) in a fresh DuckDB so both assignments share one analytical data model. Two minimal extensions adapt it to the 5-minute stream: `FACILITY` gains a `facility_code` natural key and a `network_region` column so each streamed message can be matched to the correct facility row, and a new fact table `FACILITY_POWER_EMISSIONS(obs_id, facility_id, interval_ts, power_mw, emissions_tco2e)` stores the 5-minute observations, referencing `FACILITY` by surrogate key. Integration happens at the subscriber: as each MQTT message arrives, the dashboard looks up its `facility_code` in a code-to-`facility_id` map built once from the schema and inserts the record into `FACILITY_POWER_EMISSIONS` in the shared `electricity_a2.duckdb`, so received real-time data lands in the same normalised schema as the historical Assignment 1 data. A verification join, aggregating generation and emissions by fuel category over the loaded week, confirms the fact table joins cleanly to `FACILITY` and `FUEL_TYPE` (Table 2) and quantifies the emissions concentration already noted in Section 3.

Table 2: Verification join of `FACILITY_POWER_EMISSIONS` to `FACILITY` and `FUEL_TYPE`, aggregated by fuel category over the loaded week. Energy is $\text{SUM}(\text{power_mw})/12$.

Fuel category	Facilities	Energy (MWh)	Emissions (tCO ₂ e)
Fossil	67	2,299,366	2,149,969
Renewable	229	1,144,593	101
Storage	57	173,931	0

5 Data Visualisation, Insights and Conclusions

The dashboard (Figure 3) is a single-screen Plotly Dash view that refreshes on a 1.5-second timer while the MQTT subscriber updates shared state on a background thread under a `threading.Lock`. A live map carries one marker per facility at its real coordinates, colour- and size-encoded by the selected metric. A metric toggle switches the entire view — map, KPI tiles, generation mix and leaderboard — between *power output (MW)* and *CO₂ emissions (tCO₂e)*, immediately separating the “high output, low emissions” renewable hubs from the small number of emissions-heavy thermal plants. Network-region and fuel-technology dropdowns filter every chart in lock-step; a click on any marker

opens a detail panel with the station’s name, fuel type, region, latest power and emissions, and registered capacity. Five KPI tiles report total generation, total emissions, clean-energy share, messages received, and the current NEM price and demand. Below the map, three time-series panels modelled on the official OpenElectricity Explorer give the rolling-window context a single “now” snapshot cannot: a stacked area of generation mix by fuel group, a system-wide emissions intensity line, and a twin-axis price and demand chart over the trailing six hours.

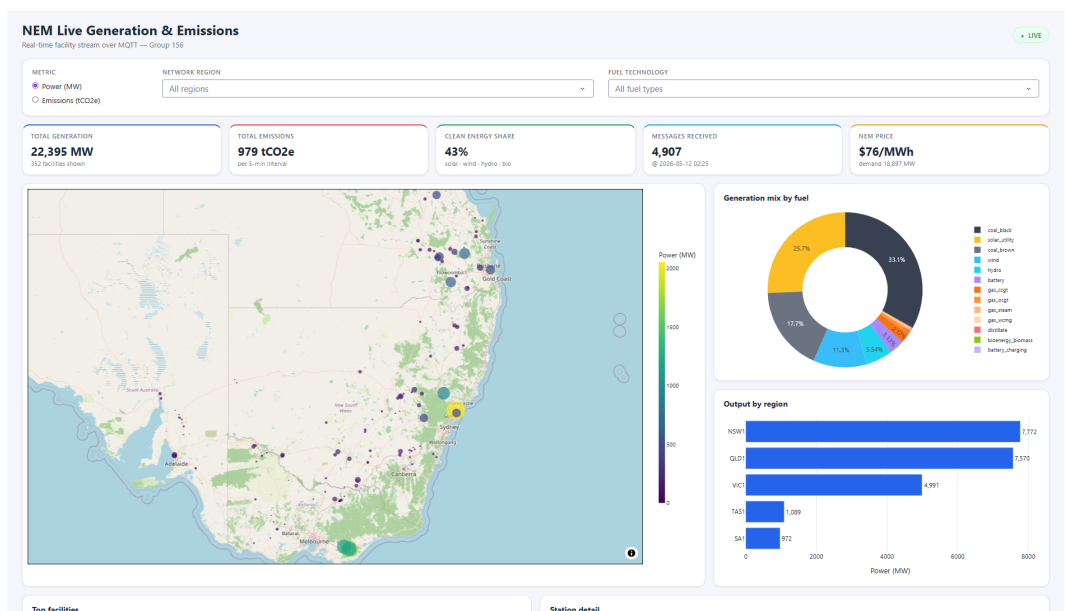


Figure 3: Live NEM dashboard. Metric, region and fuel filters apply uniformly across the map, the generation-mix donut, the by-region bars and the leaderboard; the time-series row below the map (not pictured) renders the rolling stacked area, emissions intensity and price/demand charts.

The week’s data tells a consistent story across these views. Coal supplies roughly 58% of energy but *about 95%* of emissions; renewables — wind, hydro and utility solar — supply close to a third of generation but a negligible share of emissions; and intensity swings visibly through the day as solar displaces gas around midday. The challenges we encountered were largely shape-related: facility series are keyed by unit rather than facility, so we rebuilt the unit-to-facility map and summed correctly; the 500-request daily budget forced us to batch 25 codes per call and cache aggressively; network-local timestamps and inconsistent metric names had to be normalised to a single UTC, canonical schema; concurrency required the subscriber to update shared state under a lock on paho’s network thread while Dash polls it on a 1.5-second timer; and the DuckDB write-lock means only one process may write at a time, so the publisher closes its connection before the dashboard attaches. Looking forward, the natural next steps are an incremental “latest interval” retrieval inside the continuous loop so the system can keep up with genuinely new data, QoS-1 publishing with a persistent session for guaranteed delivery, batched DuckDB inserts for higher write throughput, and — addressing the Assignment 1 feedback — ingesting the CER committed and probable registers alongside the historical accredited list to extend the renewables view.

Team contributions.

Member 1 (540989498) led data retrieval and the OpenElectricity API client (batching, caching, rate-limit handling) and the cleaning and consolidation pipeline.

Member 2 (540929966) built the MQTT publisher — ordered streaming, 0.1 s pacing, high-water-mark idempotence, and continuous execution.

Member 3 (540903678) developed the Dash map dashboard (subscriber, live map, filters, click pop-ups, the three time-series panels) and the Assignment 1 schema integration in DuckDB. All members contributed equally to the EDA, the requirements file and this report.

Appendix

Supporting detail; not counted toward the five-page report limit.

A. Task-to-artifact map

Task	Implementation	Output artifact
1. Data Retrieval	OpenElectricityClient, batched fetch	raw_openelectricity/batch_*.json, facilities.json
2. Integration & Caching EDA	flatten → unit-sum → pivot → clean → enrich shape, distributions, mix, diurnal, intensity	consolidated_facility_5min.csv, market_price_demand_5min.csv data/output/eda_*.png
3. MQTT Publishing	FacilityStreamPublisher, 0.1 s pacing, HWM	live stream (topic in §C)
4. Schema (from A1)	DuckDB 3NF schema + new fact tables	data/electricity_a2.duckdb
5. Subscribe + Visualise	Plotly Dash app + DuckDB write-back	live map dashboard (Figs. 3, 5)
6. Continuous Execution	run_continuous_stream(), 60 s rounds	mqtt_continuous_state.json

B. How to run

- (1) Run Tasks 1–4 in the publisher notebook to build the CSV cache and electricity_a2.duckdb. (2) Run all cells in the dashboard notebook; Dash opens at <http://127.0.0.1:8050>. (3) Back in the publisher, run Task 6 (run_continuous_stream) with rounds=None for the unbounded stream. The two notebooks run in *separate kernels* and communicate only through the broker.

C. Key configuration

Parameter	Value
Network / interval	NEM / 5 min
Window	2026-05-12 – 2026-05-19 (1 week)
Batch size	25 facility codes/request (~20 calls total, budget 500/day)
MQTT broker / topic	broker.hivemq.com:1883 / comp5339/group156/electricity
QoS / message pacing / round delay	0 / 0.1 s / 60 s
Consolidated rows / facilities / regions	668,096 / 353 / 5
Dashboard refresh / rolling history cap	1.5 s / 24 h of 5-min intervals

D. Message schema (one MQTT JSON message)

event_time, facility_code, facility_name, network_id, network_region, fueltech_id, capacity_registered_mw, lat, lon, power_mw, emissions_tco2e, interval, published_at.

E. DuckDB schema additions for Assignment 2

A1 3NF schema re-implemented in full (FUEL_TYPE, CORPORATION, FACILITY, NGER_REPORT, LSRE_STATION, ABS_REGION, ABS_INDICATOR). A2 adds facility_code and network_region to FACILITY, the fact table FACILITY_POWER_EMISSIONS(obs_id, facility_id, interval_ts, power_mw, emissions_tco2e), and MARKET_PRICE_DEMAND. FUEL_TYPE carries is_renewable and a category column (Fossil / Renewable / Storage / Other).

F. EDA — additional figures

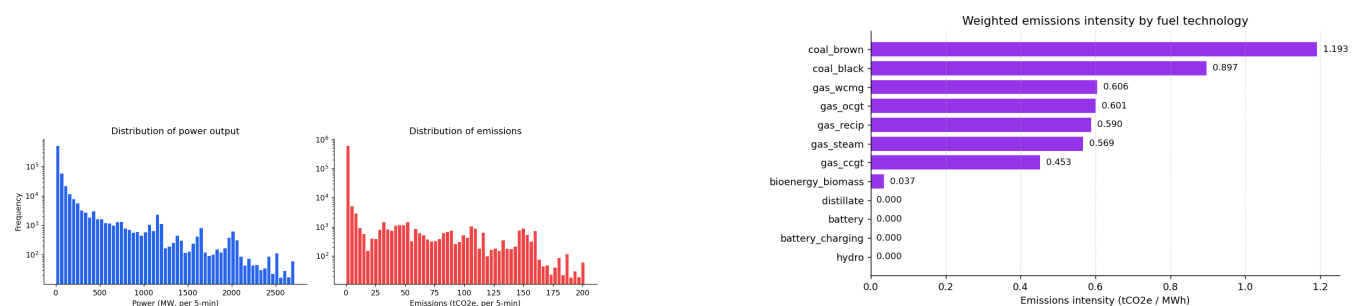


Figure 4: Left: power and emissions distributions (log axis) — heavily right-skewed with a long thermal tail. Right: weighted emissions intensity by fuel — brown coal ≈ 1.19 , black coal ≈ 0.90 , gas $0.45\text{--}0.60$, renewables ≈ 0 tCO₂e/MWh.

G. Dashboard — time-series detail



Figure 5: Dashboard lower row: top-facilities leaderboard, station pop-up (Nevertire solar), rolling generation-mix stacked area, emissions-intensity line, and twin-axis NEM price/demand over the trailing six hours.

H. Reliability and robustness

Retries: transient 429/5xx only (honouring `Retry-After`); fail-fast on 4xx. **Idempotence:** high-water-mark files prevent re-publishing; subscriber checks (`facility_id`, `interval_ts`) before inserting. **Concurrency:** shared state guarded by `threading.Lock`; DuckDB write-lock released via `con.close()` before the dashboard attaches.

I. AI Acknowledgement

The team acknowledges the use of generative AI tools, specifically Claude by Anthropic, during the preparation of this assignment. This tool was used to assist with tasks including structuring the project report and refining written explanations of technical methods. All the content and code is original and written by team members based on our own understanding of the course material. All code was tested, validated, and approved by the team before submission. The underlying analysis, data engineering decisions, and interpretation of findings are our own. Claude was mainly used for structuring and assisting in Python code (better understanding).

J. References

- Australian Bureau of Statistics. (n.d.). *Regional statistics*. <https://www.abs.gov.au/>
- Clean Energy Regulator. (n.d.). *National Greenhouse and Energy Reporting (NGER) data*. Australian Government. <https://www.cleanenergyregulator.gov.au/>
- DuckDB Foundation. (n.d.). *DuckDB documentation* [Computer software]. <https://duckdb.org/docs/>
- Eclipse Foundation. (n.d.). *Eclipse Paho MQTT Python client* [Computer software]. <https://eclipse.dev/paho/>
- OpenElectricity. (n.d.). *NEM facilities explorer*. <https://explore.openelectricity.org.au/facilities/nem/>
- OpenElectricity. (n.d.). *OpenElectricity API reference (v4)* [API documentation]. <https://docs.openelectricity.org.au/>
- Plotly Technologies Inc. (n.d.). *Dash documentation* [Computer software]. <https://dash.plotly.com/>